PRD: Financial Knowledge Graph + GraphRAG for Multi-Hop Questions

Author: Rishang | **Status:** Draft v1 | **Context:** Project for the LSEG AI/ML Intern role (Innovation & Intelligence team)

1. Summary

This project builds a pipeline that extracts entities and relations (company, subsidiary, supplier, customer, exposure) from public financial filings and news, stores them as a knowledge graph, and answers multi-hop questions such as "Which of X's suppliers are exposed to Y?". It then measures, with a proper evaluation, whether GraphRAG beats standard vector RAG on these questions, where it wins, where it loses, and what it costs.

The deliverable is not only a working system. It is a defensible answer to a practical question a data-provider team would ask: **when is the extra cost of building and maintaining a graph worth it?**

2. Problem and Motivation

Financial questions often depend on relationships spread across many documents. A supplier named in one filing, its geographic exposure in another, and a disruption in a news article cannot be answered from a single retrieved chunk. Vector RAG retrieves semantically similar passages and tends to fail when the answer requires chaining facts across documents or aggregating over many entities.

LSEG's business is built on structured, linked financial data (entities, hierarchies, relationships). A graph-based retrieval approach is a natural fit, but the benefit over plain RAG should be measured, not assumed.

3. Goals and Non-Goals

Goals

- G1. Build an end-to-end pipeline: ingest documents, extract triples, build a graph, retrieve, answer, with citations back to source passages.
- G2. Build a multi-hop question set with verified ground truth, stratified by hop count and question type.
- G3. Compare at least three systems: vector RAG, GraphRAG, and a hybrid (graph + vector).

- G4. Produce an error analysis that explains failures (extraction, entity resolution, retrieval, reasoning), not just accuracy numbers.
- G5. Report cost and latency, including one-time graph build cost and per-query cost.
- G6. Ship a small demo and clear documentation so another team could reproduce it.

Non-Goals

- Training or fine-tuning a large model from scratch.
- Real-time ingestion or a production-grade deployment.
- Using proprietary LSEG data. Only public data is used.
- Investment advice or forecasting. The system answers factual relationship questions only.

4. Users and Use Cases

The primary "users" are an analyst or data-operations person who needs relationship answers quickly, and the hiring team evaluating the project.

Use case	Example question	Hops
Direct lookup	Who are the main suppliers of Company A?	1
Chained relation	Which subsidiaries of Company A operate in Region R?	2
Multi-hop exposure	Which of A's suppliers are exposed to Event/Region/Commodity Y?	2-3
Aggregation	Which companies share more than one supplier with A?	2-3
Path/explanation	How is Company A connected to Company B?	2-4

5. Scope

In scope

- A focused universe of roughly 50-100 companies in one or two connected sectors (for example, semiconductors and electronics manufacturing), so supplier and customer links overlap.
- Sources: SEC EDGAR 10-K filings (business, risk factors, Exhibit 21 subsidiary lists) and a bounded set of public news articles about the same companies.

- Relation types: SUBSIDIARY_OF, SUPPLIER_OF, CUSTOMER_OF, COMPETITOR_OF, OPERATES_IN (region/country), EXPOSED_TO (risk factor, commodity, region, event).
- Question answering in English, with source citations.

Out of scope

- Full market coverage, multilingual documents, images, and charts.
- Temporal graph reasoning beyond tagging each edge with its source date.

6. System Overview

1. **Ingestion:** Download and clean filings and news. Parse text and tables; keep document, section, and date metadata.
2. **Chunking:** Section-aware chunks with overlap, used by both pipelines so the comparison is fair.
3. **Extraction:** An LLM extracts (head, relation, tail, evidence span, confidence) triples under a fixed schema with a structured-output format.
4. **Entity resolution:** Normalize names and aliases (embedding similarity plus rules, with an LLM check for ambiguous pairs). Link to canonical entities.
5. **Graph store:** Property graph (Neo4j, or NetworkX for a lighter setup) with edges carrying source, evidence, date, and confidence.
6. **Retrieval systems:** Vector RAG, GraphRAG, and Hybrid (see section 7).
7. **Answer generation:** The LLM answers from retrieved context only and returns the answer with citations.
8. **Evaluation harness:** Automated runs across systems and question sets, with logged retrieval, answer, tokens, latency, and cost.
9. **Demo:** A simple UI showing question, retrieved subgraph or passages, answer, and sources.

7. Systems Under Comparison

System	Retrieval	Purpose
B0: Zero-shot LLM	None	Lower bound; measures parametric knowledge and hallucination
B1: Vector RAG	Dense embeddings + reranker over chunks	Standard baseline

System	Retrieval	Purpose
S1: GraphRAG	Entity linking on the question, k-hop subgraph traversal, evidence passages attached to edges	Main system
S2: Hybrid	Graph traversal to find candidate entities, vector search to fetch supporting text	Expected best of both
Optional: Microsoft GraphRAG (community summaries)	Community-based global/local search	External reference point, if time permits

All systems use the same LLM, the same chunks, and the same prompt template for the final answer so that the retrieval method is the only variable.

8. Data

- **Filings:** SEC EDGAR 10-K documents (public, free). Exhibit 21 gives clean subsidiary lists, useful as partial ground truth.
- **News:** A bounded public corpus about the chosen companies. Exact source to be fixed in week 1 based on licensing and ease of access.
- **Question set:** About 150-200 questions.
  - Generated from graph templates (high volume, known answers), then paraphrased by an LLM for natural phrasing.
  - A manually verified gold subset of about 50 questions, checked against the source documents.
  - Stratified by hop count (1, 2, 3+) and type (lookup, chain, aggregation, path).
- **Extraction gold set:** About 100 manually labeled passages to measure triple precision and recall.

Risk to handle: Template-generated questions can favor the graph because the graph produced them. The manually verified subset and a set of questions written independently of the extraction schema are used to check for this bias, and the report will say so explicitly.

9. Functional Requirements

ID	Requirement	Priority
FR1	Ingest and parse filings and news with document and section metadata	P0
FR2	Extract triples under a fixed schema with evidence spans	P0
FR3	Resolve entities and aliases to canonical IDs	P0
FR4	Store the graph with provenance on every edge	P0
FR5	Implement vector RAG baseline	P0
FR6	Implement GraphRAG retrieval (entity linking + k-hop traversal)	P0
FR7	Implement hybrid retrieval	P1
FR8	Return answers with citations to source passages	P0
FR9	Evaluation harness with reproducible runs and logged cost/latency	P0
FR10	Demo UI showing retrieved evidence	P1
FR11	Microsoft GraphRAG reference run	P2
FR12	Simple abstain behavior when evidence is insufficient	P2

10. Evaluation Plan

Answer quality

- Answers are mostly entity sets, so use set-based precision, recall, and F1 against gold answers, plus exact match for single-entity answers.
- LLM-as-judge for free-form answers, validated against human judgments on a sample. Report agreement.
- Breakdown by hop count and question type. This is the key chart: does the GraphRAG advantage grow with hops?

Retrieval quality

- Supporting-fact recall: the share of gold evidence facts present in the retrieved context.
- Context precision: the share of retrieved context that is relevant.

Faithfulness

- Citation correctness: the cited passage actually supports the claim.
- Hallucination rate: answers containing entities absent from the retrieved context.

Graph quality

- Triple precision and recall on the extraction gold set, per relation type.
- Entity resolution precision and recall on a labeled alias sample.

Efficiency

- Tokens, dollar cost, and latency per query.
- One-time graph construction cost (extraction tokens and time).
- Break-even: the number of queries at which GraphRAG's build cost is justified, if it is.

Ablations

- Traversal depth (1, 2, 3 hops).
- Extraction model size (large vs small model) and its effect on downstream accuracy.
- With and without entity resolution.

11. Success Criteria

- Working pipeline and demo that run end to end on the chosen universe.
- Evaluation across at least B0, B1, S1, and S2 on the full question set, with confidence intervals or bootstrap estimates.
- Hypothesis tested and reported honestly: GraphRAG or hybrid improves F1 on 2+ hop questions over vector RAG. A negative or mixed result is acceptable if the analysis explains why.
- Error taxonomy covering extraction, entity resolution, retrieval, and reasoning failures, with counts.
- Reproducible repository with a README, a documented schema, and a short technical report.

12. Milestones (assumes about 6 weeks)

Week	Deliverable
1	Fix company universe, data sources, and relation schema. Download and parse documents. Set up the repository and experiment tracking.
2	Vector RAG baseline running. Extraction prompts and structured output working. Start the extraction gold set.
3	Full extraction run, entity resolution, graph loaded with provenance. Extraction metrics computed.
4	GraphRAG and hybrid retrieval implemented. Question set built and gold subset verified.
5	Full evaluation runs, ablations, cost and latency measurement, error analysis.
6	Demo, report, cleanup, and presentation. Buffer for slippage.

If the timeline is shorter, cut to a smaller universe (about 30 companies), drop the optional Microsoft GraphRAG run, and keep the hybrid system as P1.

13. Tech Stack (proposed)

- **Language and orchestration:** Python, with a thin custom pipeline (avoids hiding retrieval behavior behind a framework).
- **LLMs:** API models for extraction and answering; a smaller model as an extraction ablation.
- **Embeddings and reranking:** An open embedding model plus a cross-encoder reranker.
- **Vector store:** FAISS or Chroma.
- **Graph store:** Neo4j or NetworkX.
- **Evaluation and tracking:** Pandas, a simple experiment log (MLflow or Weights & Biases), and versioned prompts.
- **Demo:** Streamlit or Gradio.

14. Risks and Mitigations

Risk	Impact	Mitigation
Noisy extraction produces a poor graph	GraphRAG underperforms for reasons unrelated to the method	Fixed schema, evidence spans, confidence thresholds, extraction gold set, and per-relation error analysis
Entity resolution errors split or merge companies	Broken multi-hop paths	Alias rules + embedding match + LLM check on ambiguous cases; measure and ablate
Question set biased toward the graph	Inflated GraphRAG results	Independent question subset, human-verified gold set, explicit discussion of bias
Ground truth is expensive to verify	Smaller gold set	Prioritize a smaller verified set over a large unverified one; use Exhibit 21 as a free check
LLM API cost overruns	Incomplete experiments	Cap universe size, cache all LLM calls, use a smaller model for bulk extraction
Scope creep	Unfinished project	Fixed P0/P1/P2 priorities and a hard freeze on schema after week 2
Source licensing for news	Cannot share data	Use filings as the core corpus; keep news small and publish only IDs or URLs

15. Deliverables

1. Code repository with reproducible pipeline and configuration.
2. Knowledge graph snapshot and schema documentation.
3. Question set and gold annotations (subject to source licensing).
4. Evaluation results, ablations, and error taxonomy.
5. Short technical report (4-6 pages) written for both technical and business readers.

6. Demo, plus a 5-minute walkthrough.

16. Open Questions

- How many weeks are available, and is the project solo or with others?
- Is there access to a GPU, or will everything run on API models?
- Which sector pair gives the best supplier overlap in public filings?
- Which news source can be used with clean licensing?
- Which LLM and embedding model fit the budget?